
CONNECTHUB+

Product Documentation

Event management platform — administration panel, attendee mobile apps, roles & approval workflow, and QR check-in integration.

Audience: partners and integration teams

Version: 1.0 · **Date:** 2026-08-15

Platform: Web panel · iOS · Android

Contents

- 1 Administration Panel

- 2 Roles, Permissions & Event Approval Workflow

- 3 Mobile App (Attendees)

- 4 Integrations & QR Check-in API

Administration Panel

The Administration Panel is the web back office of the platform. It is where an organizing institution builds and runs everything that its audience later sees in the mobile app: the venues and halls where events take place, the event catalogue itself, ticket pricing and discounts, speakers, certificates, badges, and the reports that show how each event performed.

The panel is multi-tenant. Each institution works only with its own data, while the platform operator (superadmin) can supervise every institution from a single sign-in and switch between them with a selector in the top bar. What each person can see and do is driven by the roles assigned to their account, so the navigation menu only ever shows the modules that person is entitled to use.

The interface is available in **English, Spanish, French and Portuguese**, with a light and a dark theme. The chosen language applies to the whole panel — including dates, currency formatting and the column headings of every **Excel export**.

Access, sign-in and roles

Users sign in with their email address and password. Accounts created by an administrator receive a temporary password and are asked to set a new one the first time they sign in. Signing out is available from the avatar menu in the top-right corner, together with the language selector and the light/dark theme switch.

Permissions are granted through roles. A user can hold several roles at once, and the panel combines them.

Role	What it is for	Modules it unlocks
Platform superadmin	Operator of the platform; supervises all institutions	Everything, plus the Institutions module and the institution selector
System	The institution's master account	All modules of its institution, including Users, Activity log and Integrations
Administration	Day-to-day management of events and operations	Operations, Events (including publishing), Reports, Integrations
Finance	Financial follow-up	Finance / Payments
Operations management	Manages the physical spaces	Operations, Events, and approval/relocation of halls
Events	Creates event proposals	Events list and calendar only; every event created goes through the approval workflow

Two rules are worth remembering: the *System* role can only be granted by the platform superadmin, and a user holding only the *Events* role can never publish an event on their own — their proposals must be approved by someone else.

Conventions used throughout the panel

- **Institution selector** (platform superadmin only): a drop-down in the top bar filters every screen — users, events, finance, reports — to one institution, or to all of them at once.
 - **Inline editors**: lists open their create/edit forms directly on the page instead of navigating away, so the list stays visible while editing.
 - **Confirmation dialogs**: destructive actions always ask for confirmation, and irreversible ones say so explicitly.
 - **Images**: logos, floor plans, hall photographs, event covers and speaker portraits are uploaded from the same control (PNG, JPG or WebP). Thumbnails can be clicked to open a full-size viewer, and an uploaded image can be replaced or removed at any time.
 - **Excel export**: every reporting screen has a "Download Excel" button. The generated workbook is split into meaningful sheets and the file is named with the export date.
-

Home

The landing screen greets the signed-in user, states whether they hold full platform access or which roles they have, and shows a card for each module available to them. It works as a shortcut board: one click takes the user into Administration, Finance, Operations or Events. Users who only have a narrow role simply see fewer cards.

Administration

Users

Purpose. Manage the staff accounts of the institution — the people who work inside the panel. Attendees who register through the mobile app are not managed here.

What you see. A table with the login account, full name (with the optional group or department shown as a tag), assigned roles, status (active/inactive) and, for the platform superadmin, the institution the account belongs to.

Actions. Create a user (login email, first and last name, optional group, roles and — for the superadmin — the institution). The system generates a temporary password and emails it; if the email cannot be sent, the password is displayed once on screen so it can be handed over manually. Existing users can be edited (name,

contact email, group, roles, password reset), deactivated to block access without losing history, reactivated, or deleted outright.

Who can use it. The institution's *System* account and the platform superadmin.

Activity log

Purpose. An audit trail of what happened in the panel: sign-ins and failed sign-in attempts, records created, updated or deleted, approvals, publications, rejections, suspensions, role changes and errors.

What you see. A chronological table with date, user, action, a plain-language description of what was done (for example "approved the venue for event #262" or "changed the roles of a user"), the resulting status and the originating IP address. Clicking a row expands the technical detail of the operation.

Actions. Filter by action type, by user, by date range and — for the platform superadmin — by institution. Three quick filters isolate approvals, sign-ins or errors. Results are paginated, and the current view can be exported to Excel.

Who can use it. The institution's *System* account sees its own institution's activity; the platform superadmin sees every institution.

Integrations

Purpose. Issue API keys so the institution's own application — typically a door-scanning app — can validate the QR ticket of an attendee at the entrance.

What you see. The list of issued keys with their name, visible prefix, status (active/revoked), who created them, creation date, last use and total number of uses. The full key is displayed only once, at the moment it is generated.

Actions. Generate a new key (giving it a descriptive name such as "QR reader — main entrance"), copy it, and revoke any key that is no longer trusted. A built-in, expandable documentation panel shows the endpoint to call, the authentication header, a ready-to-copy example request, and the three possible answers: access allowed (with the participant's and event's details), ticket already used, and access denied. A second endpoint is documented for checking a ticket without registering the entry.

Who can use it. The *System* account and *Administration*.

My institution

Purpose. Maintain the institution's own profile and its payment configuration.

What you see. The institution logo, its name, address, city and country, plus a read-only **connection code** — the code attendees enter in the mobile app to link themselves to this institution. Below that, the payment configuration: payment provider, environment (test or production) and the provider's credentials.

Actions. Edit the profile and upload or replace the logo. Choose the payment provider (Nuvei, PayPal, PayPhone, Stripe, Square or Authorize.Net); the form then asks exactly for the fields that provider requires. Credentials are write-only: existing values are never displayed back, each field simply indicates whether it is already configured, and typing a new value replaces it.

Payments themselves are captured in the mobile app through the configured provider. The panel stores the configuration and reports on the results; it does not process card data.

Who can use it. The *System* account of the institution.

Institutions (platform superadmin)

Purpose. Onboard and supervise the institutions that use the platform.

What you see. Every institution with its logo, name, city and country, number of users and status: pending, approved, rejected or suspended.

Actions. Register a new institution; approve a pending one — which asks for the email of its future *System* user and then creates that account and shows its temporary credentials once; reject, suspend or reactivate an institution; open and edit any institution's profile (including its payment configuration); and delete an institution. Right after approval, the panel opens the new institution's profile so its payment credentials can be completed immediately.

Who can use it. The platform superadmin only. Other users see "My institution" in this place.

Finance — Payments

Purpose. Follow the money collected by the institution's events.

What you see. Three headline figures — total collected, number of payments and number of free registrations — followed by a bar chart of revenue per event, a table of revenue and payment count by event, the same broken down by month, and a list of the most recent payments showing event, amount, payment method with the last four digits of the card, and date.

Actions. Filter by event, month and year (filters combine, and can be cleared in one click), and export the complete view to Excel as a four-sheet workbook: summary, by event, by month, and payments.

Who can use it. The *System* account and users holding the *Finance* role.

Operations — Venues, halls, sub-halls and layouts

Purpose. Describe the physical infrastructure that events are booked into, and set the rental price of each space.

Venues

What you see. The list of venues with their floor-plan thumbnail, name, location, owning institution (for the superadmin) and number of halls.

Actions. Create, edit and delete venues; upload a floor plan or site map; open a venue to manage its halls.

Halls

Inside a venue, each hall is shown as a card with its photograph or diagram, name, maximum capacity, rental price per day, a "subdividable" marker, and counters of its sub-halls and layouts.

Actions. Create and edit halls (name, maximum capacity, price per day, subdividable flag), upload an image, delete a hall, or open its management panel — which has two tabs.

Sub-halls

For subdividable halls, each partition is defined with its own name, capacity, price and image. Sub-halls can be edited inline and removed.

Layouts

A layout is a named combination of sub-halls — for example "Auditorium A+B" — used when a hall is opened up or split for a particular event. Each layout has its own name, its selection of sub-halls, an optional diagram, and an active/inactive switch so seasonal configurations can be retired without deleting them.

Who can use it. *System, Administration and Operations management.*

Events

The Events area covers the full life of an event: proposal, space assignment, approval, publication, content, and the material handed to attendees.

Approval workflow

Every event moves through two steps before it becomes visible in the mobile app:

1. **Venue approval** — whoever manages the halls (*Operations management* or *System*) confirms the requested space, or relocates the event to a different hall or set of dates. A relocation counts as a resolved venue.
2. **Publication** — *Administration* or *System* publishes the approved event, making it available to attendees.

At any pending step the event can be **rejected** with a mandatory reason. A published event can be **suspended** — withdrawn from the app with an optional reason, while keeping its registrations and payments intact — and later **republished**. Each event carries a coloured status badge, a progress indicator of the two steps, and the name and timestamp of whoever approved, published, rejected or suspended it. An expandable history shows every space request, relocation, approval, publication, rejection and suspension with its author and date.

Event list

What you see. A table of events with cover thumbnail, title, badges (workshop, approval status), the parent event when the entry is a workshop, the rejection or suspension reason when applicable, the event days and

times (including preparation and cleanup allowances), the assigned space, price or "free", the number of registrations, and whether the event is featured.

Actions. Create an event; open the read-only detail view; edit; delete; toggle "featured"; and — depending on role — approve the venue, approve publication, reject, suspend or republish directly from the row. A shortcut leads to the calendar view.

Who can use it. All roles with access to Events; the available buttons differ by role.

Calendar

Purpose. See the whole programme at a glance and spot free days.

What you see. A month grid where each event appears on every day it actually runs, colour-coded by hall, with a per-day counter. A side panel details the selected day: each event's schedule, the real occupancy window once preparation and cleanup are included, venue and hall, layout or sub-halls used, number of registrations and price.

Actions. Move between months, jump to today, click an event card to open it for editing, or create a new event on the selected day with the date pre-filled.

Reservations

Purpose. Book a hall for internal use without publishing anything to the audience.

What you see. A monthly occupancy calendar filtered by venue and hall, where each day lists what is already booked and distinguishes public events from private bookings. Selecting a day shows the existing bookings — or confirms the day is free.

Actions. Create a private booking with a title and a start and end time; or switch to full event creation if the booking should become a real, published event.

Who can use it. All Events roles except the restricted *Events* role, which is limited to the list and the calendar.

Hall approvals

Purpose. A single queue for everything waiting on a decision.

What you see. Two sections — requests pending venue approval, and events whose venue is resolved and are ready to publish. Each row shows the event title, the requested space in full (venue, hall, layout or sub-halls), the days and times, who requested it, their group, and the institution.

Actions. Approve the venue, reject with a reason, approve publication, or open the event to adjust its hall and dates before deciding.

Who can use it. *System*, *Administration* and *Operations management*, each seeing the actions their role allows.

Event editor

The editor gathers the whole event definition on one page, with the heavier content areas collapsed until they are needed.

Basics. Title, description, an optional parent event (which turns the entry into a workshop of a larger event), and an item code for accounting cross-reference.

Days and schedule. An event is defined as a list of days, each with its own start and end time — so a programme that runs on non-consecutive dates is described exactly as it happens. Days can be added and removed freely.

Space. Choose the venue and then the hall — or the whole venue. When a hall is chosen, the space can be reserved as the complete hall, as a specific layout, or as an individual sub-hall. Reference images of the venue, hall, layout and sub-hall are displayed next to the selection, and the editor shows an occupancy agenda for the selected days: which other events already sit in that space, at what times, and what window they truly occupy once preparation and cleanup are counted.

Commercial and capacity settings. Ticket price, an optional VAT-included flag with its percentage, preparation and cleanup minutes, expected audience, a "featured" flag with an ordering number for the app's highlights, and a "private" flag for events that must not be published. Users restricted to the *Events* role see a notice that their event will go to review instead of the private switch.

Cover image. Upload, replace or remove the image shown in the mobile app.

Discount coupons. Create coupons with a code and either a fixed amount or a percentage discount, optionally limited to a maximum number of uses. The list shows how many times each coupon has been used against its limit, and coupons can be deleted.

Extended detail. The richer content the app displays on the event page: short and long description, "what you will learn", topics and prerequisites as editable bullet lists, level (beginner, intermediate, advanced), modality (in person, online, hybrid), duration with its unit (hours, weeks, sessions, days), and the certificate policy — whether certificates are issued, of which kind (attendance, participation, completion) and how they are delivered (download, email, in-app).

Speakers. Add each speaker with full name, job title, organization, role in the event (speaker, co-speaker, moderator or keynote), biography, portrait photo and a "featured" flag. The list shows avatars and role badges, and entries can be edited or removed.

Certificates. A visual designer for the event's certificate: upload a template image, switch on the fields to print over it — participant name, event name, date, certificate type, time, institution and verification code — and drag each one into place on a live preview. Font size, colour and bold can be adjusted per field. Once the template is saved, the attendee list is shown with who attended and who already has a certificate, and certificates can be generated for a selected group or for everyone who attended in a single batch.

Badges. From the same area, a printable badge sheet opens for the event. It lists every participant with their name, the event name and their check-in QR code. Attendees can be searched by name, ID number or email, and selected individually or all at once. Three print templates are available — **Classic** (horizontal badge with QR), **ID Card** (vertical credential with the institution band, initials, QR and the event's validity dates) and

Compact (minimal horizontal label) — each with a preview. Printing outputs only the selected badges, sized for cutting, with the panel's own interface excluded from the page.

Reports

Attendance

Purpose. Measure participation across events.

What you see. Headline figures for number of events, registrations, attendees, no-shows, pending check-ins and the resulting attendance rate; a stacked bar chart comparing attendance, no-shows and pending per event; and a table with, for each event, its date, expected audience, registrations, attendance, no-shows and attendance rate.

Actions. Filter by year, by any combination of months, and by a single event. Drill into an event to list its registrants with name, email, phone and attendance status. Both the report and the drill-down list export to Excel — the report as a summary sheet plus a per-event sheet, the drill-down as an attendee sheet that also carries the registration and entry timestamps.

Who can use it. *System and Administration.*

Hall usage and revenue

Purpose. Show how intensively the physical spaces are used and what they earn.

What you see. Headline figures for rental revenue, ticket revenue, number of bookings, days booked, halls used out of the total, and occupancy rate; a ranking chart of the busiest halls; a per-hall table with bookings, days booked, occupancy rate, rental revenue and ticket revenue; and a day-by-day table of halls used, bookings and rental revenue.

Actions. Filter by year, by months and by venue, and export to Excel as a three-sheet workbook: summary, by hall, and by day.

Who can use it. *System and Administration.*

Feedback

Purpose. A direct channel between the institutions using the panel and the team that maintains the platform.

What you see. Institution users see the form for submitting a suggestion, a problem report or a general comment, followed by their submissions with the status assigned to each and any written reply. The platform team sees every submission with its author, institution, type, date and status.

Actions. Institution users submit messages and read the replies. The platform team filters by status, moves items through *New* → *Reviewed* → *Planned* → *Done*, writes a reply that becomes visible to the person who

reported it, and exports the whole list to Excel.

Who can use it. Every signed-in user can submit and read their own thread; the platform team performs the triage.

Languages and exports at a glance

- **Interface languages:** English, Spanish, French and Portuguese, switchable at any moment from the avatar menu without losing the current screen. Dates, month names and currency amounts follow the selected language.
- **Themes:** light and dark.
- **Excel exports available in:** Activity log, Finance (summary, by event, by month, payments), Attendance report (summary, by event) and its attendee drill-down, Hall usage report (summary, by hall, by day) and Feedback. Every workbook is generated in the language currently active in the panel.

Roles, Permissions & Event Approval Workflow

This section explains **who can do what** inside the platform and **how an event travels from an internal request to a published listing in the mobile app**. It is written for an institution that is evaluating or adopting the product: no technical setup is required to understand it.

Roles & Permissions

1. The model in one paragraph

Every administrative user belongs to **one institution** (the tenant) and carries **one or more roles**. Roles are additive: a user with two roles gets the union of both sets of permissions. Above the institutions there is a single platform-level **superadministrator** used by the platform operator, not by customers. Inside an institution, an optional **group** label lets a large organization split its event teams (faculties, departments, business units) without creating separate accounts or separate tenants.

Attendees who use the mobile app are a completely separate audience: they never receive an administrative role and never access the web panel.

2. The five roles

SYSTEM

The institution's owner-level role. It is the only role that can administer **users and access**: create accounts, edit them, activate or deactivate them, and assign roles. It also has visibility over every other module of its institution — venues, events, approvals, finance, reports and integrations — and it is the only institution role that can open the **activity log (audit trail)**, restricted to its own institution.

- Sees: everything belonging to its institution.
- Typical holder: the platform administrator inside the customer organization.
- Notes: the SYSTEM role itself can only be granted by the platform superadministrator, so an institution cannot silently escalate itself.

ADMINISTRATION

The institutional manager. Full visibility of every event in the institution (not limited to the events it created), access to the venue/space catalogue, the approvals queue, event publication, reports and API integrations. It does **not** manage users and does **not** see the financial module unless the FINANCE role is added on top.

- Sees: all events, all venues, reports, approvals.
- Decides: the **final publication** of an event (and its suspension / re-publication).

FINANCE

A narrow, read-oriented role for the finance team. It opens the financial module — revenue collected per event and per period for its institution — and nothing else. It does not create events, does not touch venues and does not approve anything.

OPERATIONS MANAGEMENT

The owner of the **physical space**. It administers venues, halls, sub-halls, subdivision layouts, floor plans and reference images, and it is the role that decides **which space and which dates an event actually gets**. In the approval workflow it approves the requested venue as it stands, or relocates the event to a different space/date and then approves it. It does not publish events to the mobile app.

EVENT

The event creator — deliberately the most restricted role. A user with only this role can create an event, describe it (agenda, speakers, images, price, coupons, capacity) and **request** a space and dates, but:

- it only sees **its own events**: those it created plus those of its **group**;
- it cannot mark an event as public by hand — publication is never in its hands;
- every edit it makes sends the event back to the beginning of the approval cycle;
- it reads the venue catalogue and the availability calendar (to pick a space) but cannot modify it;
- it does not see finance, user management, the audit trail or private-booking screens.

3. The platform superadministrator

The superadministrator is a **platform-operator capability**, not an institution role. It exists to onboard and supervise customers:

- it works across **all institutions** and can filter any screen by institution;
- it registers, approves, rejects, suspends and reactivates institutions;
- it is the only actor that can grant the SYSTEM role;
- it sees the **complete audit trail** across every institution;
- when it writes data it must explicitly state which institution the record belongs to, so cross-tenant writes are always deliberate.

4. Groups: separating teams and areas inside one institution

A single institution frequently has several independent event teams — a university with faculties, a chamber of commerce with committees, a company with business units. The **group** is a free-text label attached to a user (for example **Engineering School**, **Marketing**, **Continuing Education**) that solves this without multiplying tenants:

Behaviour	Effect
Inheritance	Every event created by a user inherits that user's group automatically.
Visibility	A user with only the EVENT role sees the events it created plus every event of its own group — and nothing else.
Collaboration	Colleagues in the same group can pick up and continue each other's events (edit, images, speakers, coupons) without an administrator intervening.
Approvals	The group is displayed next to the requester in the approvals queue, so the approver immediately knows which area is asking for the room.
Boundaries	The group narrows visibility, it never widens it. It never crosses institutions, and it never hides anything from SYSTEM, ADMINISTRATION or OPERATIONS MANAGEMENT, who always see the whole institution.
Optional	Leaving the group empty is perfectly valid: the user then simply sees only the events it created.

5. Access summary by module

Legend: ✓ full access · ● partial / read-only or scoped · — no access.

Module	SYSTEM	ADMINISTRATION	FINANCE	OPERATIONS MGMT	EVENT	Superadmin
Users & role assignment	✓	—	—	—	—	✓
Institution profile	✓	🔒	—	—	—	✓ (all institutions)
Institution registry (approve / suspend customers)	—	—	—	—	—	✓
Activity log / audit trail	🔒 (own institution)	—	—	—	—	✓ (all)
API keys & integrations	✓	✓	—	—	—	✓
Finance — revenue & payments	✓	—	✓	—	—	✓
Venues, halls, sub-halls, layouts, maps	✓	✓	—	✓	🔒 (read-only, to pick a space)	✓
Availability calendar	✓	✓	—	✓	🔒 (own scope)	✓
Private bookings / reservations	✓	✓	—	✓	—	✓
Events — list & calendar	✓ (all)	✓ (all)	—	✓ (all)	🔒 (own + group)	✓ (all)
Create / edit an event	✓	✓	—	✓	🔒 (own + group, goes to approval)	✓
Approvals queue	✓	✓	—	✓	—	✓
Approve the venue (step	✓	—	—	✓	—	✓

Module	SYSTEM	ADMINISTRATION	FINANCE	OPERATIONS MGMT	EVENT	Superadmin
1)						
Approve publication (step 2)	✓	✓	—	—	—	✓
Reject with reason	✓	✓	—	✓	—	✓
Suspend / re-publish a live event	✓	✓	—	—	—	✓
Feature an event in the app	✓	✓	—	✓	🕒	✓
Attendees, badges, QR check-in, certificates	✓	✓	—	🕒	🕒 (own events)	✓
Reports — attendance & venue usage	✓	✓	—	—	—	✓
Feedback / suggestions	✓	✓	✓	✓	✓	✓

6. How the rules are enforced

Permissions are not a cosmetic filter on the interface. The web panel hides what a user may not use, and **the API independently re-checks the same rules on every request**: the session identifies the user, their institution, their roles and their group; each endpoint declares which roles may reach it; and every record touched is verified to belong to the caller's institution before it is read or written. A link copied from another user, or a request crafted by hand, is rejected the same way.

Event Approval Workflow

1. Why the workflow exists

An event consumes two scarce things: a **physical space on a specific date** and the **attention of every attendee** (a published event triggers a push notification). The workflow separates those two decisions and gives each one to the team that owns it:

1. **Whoever manages the rooms decides the room** — Operations Management.
2. **Whoever represents the institution decides what goes public** — Administration.

Nothing reaches the mobile app until both have said yes.

2. Who does what

Step	Action	Who can perform it
1	Create the event and request space + dates	EVENT (and any higher role)
2	Approve the venue as requested, or relocate it and then approve	OPERATIONS MANAGEMENT · SYSTEM
3	Approve publication (goes live + push)	ADMINISTRATION · SYSTEM
Any pending step	Reject with a mandatory reason	OPERATIONS MANAGEMENT · ADMINISTRATION · SYSTEM
After publication	Suspend / re-publish	ADMINISTRATION · SYSTEM

An event created directly by ADMINISTRATION or SYSTEM does not need the workflow — those roles already hold the publication authority, so their events are published on creation (unless they explicitly mark the event as private).

3. Step by step

Step 1 — Request (EVENT user)

The organizer creates the event and requests a space: an entire venue, a full hall, one of the hall's predefined subdivision layouts, or a specific sub-hall — plus the day-by-day schedule, including optional setup and teardown buffers. The system checks the requested slot against the live calendar and refuses an overlapping request immediately.

The event is saved as a **draft**:

- it is **not visible in the mobile app**;
- **no push notification is sent**;
- the organizer can keep working on it (description, speakers, images, price, coupons, capacity);
- the organizer **cannot** make it public by hand, and any edit keeps it in draft, waiting for review;
- the request is recorded in the event's history: exactly which venue, hall, sub-hall, layout and days were originally asked for.

The request appears immediately in the **Approvals** screen of the reviewers, showing the requester, their group, the requested space and the requested days.

Step 2 — Venue decision (Operations Management)

Operations Management opens the queue and has three options:

- **Approve as requested.** One explicit *Approve venue* button. Availability is re-validated at that exact moment, because the calendar may have changed since the draft was created; if another booking has since taken the slot, the approval is refused and the conflict is reported.
- **Relocate.** Open the event and change the hall, the sub-hall, the layout or the dates — the natural way to consolidate the calendar. The move is recorded as a *before* → *after* entry in the event's history, and the event carries a **Relocated** badge so everyone downstream can see it did not get the room it asked for.
- **Reject** with a reason (see below).

Important rule: editing is not approving. Relocating an event never approves it by itself. Operations Management can rearrange the calendar as many times as needed and the event stays pending until someone presses the explicit approval button. This keeps "I moved it while reorganizing" and "I authorize this room" as two different, individually traceable decisions.

After this step the event is **venue-approved** (or **relocated**) — still hidden from the mobile app. The panel shows a confirmation line stating who approved or relocated it and when.

Step 3 — Publication (Administration / SYSTEM)

The event moves to the *Ready to publish* list. Administration reviews the public-facing content and presses **Approve publication**. Only at this moment:

- the event becomes **visible in the mobile app** for the attendees linked to that institution;
- the **push notification** announcing the new event is sent;
- registration and ticketing open;
- the panel records who published it and when.

This is the only moment a push is emitted for a workflow event, so a draft that bounced between reviewers several times never spams attendees.

Rejection (any pending step)

Operations Management, Administration and SYSTEM can reject a pending event at any point. **A reason is mandatory** and is typed into a dedicated dialog. The event:

- is marked **Rejected** and stays hidden from the app;
- returns to its creator, who sees the reason attached to the event;
- can be corrected and resubmitted — editing it as the organizer sends it back to draft and the cycle restarts from step 2.

Suspend and re-publish (after publication)

A live event sometimes has to disappear from the app — a cancelled speaker, a venue incident, a pricing mistake. Deleting is not the answer: an event that already has registrations, issued tickets or recorded payments **cannot be deleted at all**, by design. Instead, Administration or SYSTEM **suspends** it:

- it is withdrawn from the mobile app immediately;
- registrations, tickets, payments, certificates and the full history are preserved untouched;
- an optional reason can be recorded;
- **Re-publish** puts it back online without repeating the approval cycle.

4. Flow diagram



```
|| SUSPENDED ||
|| removed from the app, ||
|| NOT deleted – registrations, ||
|| tickets, payments and ||
|| history preserved ||
└─ re-publish ─┐ (no new approval cycle)
```

5. Status reference

Status	Visible in the app?	What it means	Who moves it forward
Draft	No	Created by an organizer; venue and dates requested, awaiting review	Operations Management / SYSTEM
Venue approved	No	The requested space was granted as asked	Administration / SYSTEM
Relocated	No	Operations Management moved the space or the dates and approved the new slot	Administration / SYSTEM
Published	Yes	Live in the app; push sent; registration open	—
Rejected	No	Turned down with a documented reason; back with the organizer	Organizer (correct and resubmit)
Suspended	No	Temporarily withdrawn from the app; all data preserved	Administration / SYSTEM (re-publish)

Events created before the workflow was introduced keep behaving exactly as they always did: they are treated as published unless explicitly marked private, and they can still be suspended.

6. Traceability

Two complementary records answer "who did this, and when".

a) Event history — one timeline per event. Every event carries its own chronological log of the space decisions: *requested*, *moved*, *approved*, *published*, *rejected*, *suspended*, *re-published*. Each entry stores the acting user, the timestamp and a readable detail. For requests and relocations, the detail contains a full snapshot — venue, hall, sub-hall, layout and the day-by-day schedule — resolved to names at the time it was written, so the record still reads correctly years later even if a room is renamed. A relocation stores both the **before** and the **after**, which makes "why is my event in a different room?" a one-click question to answer. Rejections and suspensions store the reason.

b) Activity log — one register per institution. Independently of the events module, the platform records every login (successful and failed) and every change made through the panel: the user, the date and time, the action, the affected area, the outcome and the originating network address. Approval actions are recorded under their own names — *approve venue, publish, reject event, suspend event, re-publish*, alongside *role change, user status change* and so on — so an auditor does not have to reverse-engineer generic "update" entries. Values that must never be stored (passwords, tokens, keys) are automatically redacted before anything is written, and the log is append-only from the users' point of view: nobody can edit or erase an entry from the interface.

The activity log is **scoped per institution**: an institution's SYSTEM user sees the movements of their own institution and only those. The platform superadministrator sees every institution, with filters by action, user and date range.

Architecture at a Glance

ConnectHub is delivered as a hosted, multi-tenant platform with three surfaces over one shared core: a **web administration panel** used by the organizing institutions, a **REST API** that holds all business logic, and **native mobile apps for iOS and Android** used by attendees. All persistent information — institutions, users and roles, venues and halls, events and schedules, registrations, tickets, payments, certificates and audit records — lives in an **Oracle database**, with images and documents kept in a dedicated file service. **Multi-tenancy** means every institution operates as an isolated workspace: its users, venues, events, attendees and reports never mix with another institution's, and attendees join an institution by entering a connection code in the app. On the security side, panel sessions and attendee sessions are two separate systems signed with separate keys; access is granted through short-lived **JWT** tokens with silent renewal, so revoking or changing a user's roles takes effect on the next renewal rather than at the next login; **role-based authorization** is declared per endpoint and re-verified server-side on every request, never only in the browser; **institution isolation** is enforced by resolving the owning institution of every record before reading or writing it; passwords are stored salted and hashed, with forced rotation for temporary credentials; and a global **audit layer** records logins and changes, redacting sensitive values, giving each institution a verifiable account of who did what and when.

Mobile App (Attendees)

The ConnectHub mobile app is the attendee-facing product of the platform. It is a native application for **iOS and Android**, published on the **App Store** and on **Google Play** (store name: *ConnectHub+*). Everything an attendee needs happens here: joining an institution, discovering its events, registering or buying a ticket, entering the venue with a QR code, following a personal agenda, collecting digital certificates, and connecting with other attendees.

The app talks to the same event catalog that institution staff manage from the web panel, so anything published by an organizer becomes visible to attendees immediately.

1. At a glance

Area	What the attendee gets
Discover	Featured carousel, full event program, quick filters, event detail with day-by-day agenda, speakers, workshops and venue
My Agenda	Personal calendar of the events the attendee holds a ticket for, as a list or a month grid
Tickets	All tickets with a scannable QR code for entry, plus access to certificates
Community	Per-event community walls, attendee directory, connection requests and private 1-to-1 chats
Profile	Account, networking profile and privacy, language, theme, institutions, account deletion

Interface languages: English (default), Spanish, French and Portuguese. The language can be switched at any time and applies instantly across the whole app, including dates and month names.

Appearance: Light, Dark or "follow the system" theme.

2. Getting started

2.1 Institution connection code

ConnectHub is a multi-tenant product: attendees do not browse a global catalog, they join the institution that invited them. On first use the app asks for a **connection code**, which the institution distributes in its invitation, ticket email, landing page or printed material.

- The code is entered once, and the app links the attendee's account to that institution.

- An invalid or unknown code produces a clear message ("we couldn't find that code") without leaving the screen.
- After the code is accepted, the attendee lands directly on the Discover tab with that institution's program already loaded.

2.2 Creating an account and signing in

The app opens on the sign-in form; creating an account is one tap away from the same screen.

Method	Notes
Email + password	First name and last name are requested at sign-up because they are printed on certificates. Passwords must be at least 8 characters and can be revealed with the eye icon while typing.
Continue with Google	Available on Android. One tap, no password to remember.
Sign in with Apple	Available on iOS devices that support it, using the official Apple button. Apple's private-relay email addresses are fully supported.

Errors are explained in plain language: wrong email or password, email already registered, or password too short.

Email verification. Accounts created with email and password must confirm their address before using the app. A verification link is sent by email, and the app shows a blocking screen with three actions: *I already verified* (re-checks the account), *Resend email* and *Sign out*. Accounts created with Google or Apple are already verified and skip this step.

2.3 Password recovery

From the sign-in screen, *Forgot password?* sends a reset link to the address typed in the email field. For privacy the confirmation message is always the same ("if an account exists for that email, we've sent a link"), so the app never reveals whether an address is registered. The new password is set on a secure web page opened from that email.

2.4 Belonging to more than one institution

An attendee can hold codes from several institutions at the same time (for example a university and a professional association).

- **My institutions** lists all of them with logo, city and country, and marks the active one.
 - A scope selector switches between **one institution** and **All my events**, an aggregated feed that mixes the programs of every institution the attendee belongs to.
 - **Add a code** at any time to join another institution without losing the existing ones.
-

3. Discovering events

3.1 The Discover home

The home screen is built for browsing a live program:

- A **top bar** showing the active institution (logo, name, city) that opens the institution switcher, plus a quick language toggle.
- **Institution chips** when the attendee belongs to more than one, to filter the feed instantly.
- A **featured carousel** of highlighted events with cover image, dates, price or "Free", and venue, with page indicators.
- Four **quick access tiles**: *Program* (everything), *Featured*, *Free* and *Tickets*.
- An **infinite list** of upcoming events with cover, institution name, title, room, dates and price, loading more as the attendee scrolls.
- Pull-to-refresh, loading placeholders, a retry action if the network fails, and a friendly empty state when an institution has not published anything yet.

3.2 Filters and search

- **Program / Featured / Free** narrow the list without leaving the screen.
- **Institution scope** (one institution vs. all of them) applies on top of that.
- The event catalog also supports **search by event name** (case-insensitive, partial matches), used to narrow long multi-day programs.

3.3 Event detail

Tapping any event opens a full page with a large cover image (tap to view it full screen) and, depending on what the organizer published:

- **Dates and price** chips, event title, organizing institution, **venue** (site and room) and start/end times.
- **About** — the long description of the event.
- **What you'll learn** — a checklist of outcomes.
- **Topics** — tag chips.
- **Agenda by day** — one row per day of the event, with weekday, day number and the schedule for that day. Multi-day congresses are shown day by day.
- **Speakers** — a horizontal gallery. Keynote speakers are badged. Tapping a speaker opens a detail sheet with photo (zoomable), role and organization, tagline, full biography, website and social links.
- **Workshops** — sessions attached to the main event, each with its dates, start time and price, navigable to their own detail page.
- **Save** — a bookmark button to keep an event handy.
- A **fixed action bar** at the bottom that always shows the price and the right call to action: *Register* — *free*, *Register* (paid) or *View ticket* if the attendee is already registered.

4. Registering and buying tickets

4.1 Free events

One tap on *Register — free* registers the attendee and takes them straight to their QR ticket. If the profile is missing a first or last name (both are printed on certificates), the app offers to complete the profile first and returns to the flow afterwards.

4.2 Paid events — checkout

Paid events open a dedicated checkout screen:

1. **Order summary** — event cover and title, subtotal, tax and total.
2. **Discount code (optional)** — the attendee types a coupon and taps *Validate*. The app answers immediately with the outcome: applied (showing the discount and the new total), invalid, or usage limit reached. A validated code is locked in and can be removed with one tap.
3. **Billing details** — before the payment window opens, the app confirms first and last name (these go on the certificate), identification document (**ID card / Passport / Tax ID** plus number, required by the payment gateway) and a **billing email**, which can differ from the account email and is where the payment receipt is sent. The fields are pre-filled from the profile and saved for the next purchase.
4. **Secure card payment** — the gateway's hosted checkout opens inside the app for credit or debit card payment. The whole screen is locked while the transaction is being confirmed, and on the web version the browser warns against reloading, so a purchase can never be duplicated by accident.
5. **Outcome** — on success the registration is confirmed, a confirmation email is sent and the attendee is taken to *My Tickets*. Pending and rejected payments are explained with their own message, and nothing is charged twice if the attendee retries.

Workshops attached to a paid main event cannot be purchased on their own: the checkout shows which main event must be bought first and offers a shortcut to it.

Card data is handled entirely by the payment gateway; the app never stores full card numbers.

4.3 My Tickets and QR check-in

The **Tickets** tab lists every ticket the attendee holds, with the event cover, title and dates, a *Workshop* badge when applicable, and a *Checked in* badge once the attendee has been scanned at the door.

Opening a ticket shows a **large, high-contrast QR code** on a white card — designed to be scanned reliably from a phone screen at the entrance — together with the event title and the ticket code in text as a fallback. This is the credential the attendee presents at check-in.

4.4 Personal agenda

The **My Agenda** tab turns the attendee's tickets into a personal schedule, in two interchangeable views:

- **List** — grouped by day, ordered by start time within each day; upcoming days first, past days below and dimmed with a *Past* label.
- **Calendar** — a month grid (weeks starting on Monday) with a dot on every day that has activity, month navigation, and the sessions of the selected day underneath.

From either view the attendee can jump to the event page or open the QR ticket directly. Events can also be bookmarked from the catalog to keep track of ones they have not registered for yet.

5. Digital certificates

When an institution confirms attendance, a **digital certificate** becomes available on the corresponding ticket.

- The certificate is rendered as a **designed image** using the template the institution configured for that event, personalized with the attendee's name.
- Every certificate carries a unique **verification code** and a public verification page, so it can be validated by a third party.
- **Add to LinkedIn** opens LinkedIn with the certification pre-filled (name, issuing organization, issue date, credential ID and verification URL), the same way industry credential platforms work.
- **Download image** saves or shares the certificate file.
- Because the attendee's name is printed on the certificate, first and last name become locked once a certificate has been issued; changes from that point must be requested from the institution.

If an event has no certificate template configured, the app still displays a branded certificate card with the same information and verification code.

6. Community and networking

Community is organized **per event**: attendees get access to the conversation of the events they hold a ticket for.

6.1 Community hub

The **Community** tab brings everything together in one place:

- **Messages** — private 1-to-1 conversations, with avatar, last message, time and an unread counter.
- **Event communities** — one entry per event the attendee has joined, with cover image, last message and number of participants.

6.2 Event community wall

Each event has its own wall: a chat-style feed where attendees, speakers and organizers post messages. It includes a message composer, chronological bubbles distinguishing the attendee's own posts, author

avatars, and automatic scrolling that never interrupts someone reading history.

- **Attendees** — the header opens the directory of participants with a public profile; tapping any of them opens their profile.
- **Leave / rejoin** — attendees can leave a community (they stop seeing and posting messages) and rejoin later at any time.

6.3 Profiles and connections

- **Public profile** — photo, name, profession, company, biography and LinkedIn link.
 - **Privacy control** — each attendee chooses between a **public profile** (anyone at their events can see them and message them) and a **private profile** (others must send a connection request first). Private profiles appear locked to strangers and are excluded from the public attendee directory.
 - **Connection requests** — *Connect* sends a request; the *Connections* screen lists incoming requests (accept or decline) and the list of accepted connections.
 - **Private chats** — *Send message* opens a 1-to-1 conversation. If the other person's profile is private and there is no connection yet, the app explains that a connection request is needed first.
-

7. Push notifications

The app registers the device for push notifications after the attendee signs in, asking for the system permission the first time.

- The main notification is the **announcement of a new event** published by any institution the attendee belongs to.
 - Notifications are shown even when the app is open in the foreground.
 - Notifications are optional: if permission is denied, or later revoked from the phone's settings, the app keeps working normally without them.
-

8. Profile and settings

The **Profile** tab is the attendee's control panel:

- **Account card** — avatar, name, email, email-verification status and *Sign out*.
- **Theme** — System, Light or Dark.
- **Language** — English, Spanish, French, Portuguese.
- **My profile** — a read-only view of the networking profile with a visibility badge, an "incomplete profile" hint, and a *See how others see you* preview.
- **Edit profile** — profile photo (taken from the device gallery, with permission handling), first and last name, identification document, billing email, profession, company, biography, LinkedIn URL and the public/private visibility switch.

- **Connections** — pending requests and existing connections.
 - **My institutions** — switch the active institution or add a new connection code.
 - **Privacy policy** — direct link to the published policy.
 - **Delete account** — see below.
-

9. Account deletion

The app includes self-service account deletion, as required by both app stores:

1. *Profile* → *Delete account* opens a destructive confirmation dialog stating exactly what happens.
2. On confirmation, the attendee's **profile, connections, conversations and saved payment methods are permanently removed**.
3. **Payment and ticket records are retained in anonymized form**, because institutions need them for their own accounting and attendance records. They can no longer be linked back to the person.
4. The action is irreversible, and the attendee is returned to the sign-in screen.

The same deletion request can also be started from a public web page, for people who no longer have the app installed.

10. Practical notes

- **Session handling** — the attendee stays signed in between uses; credentials are stored in the device's secure keychain. If a session expires, the app returns to the sign-in screen without losing any data.
- **Always fresh** — every list refreshes when it comes back into view and supports pull-to-refresh, so changes made by organizers in the web panel (a new speaker, a rescheduled day, a new workshop) appear without reinstalling or signing out.
- **Resilient by design** — loading placeholders, explicit error states with a retry action, and helpful empty states are used throughout instead of blank screens.
- **Demo access** — a demo connection code can be provided to reviewers or prospective clients so they can explore the full attendee experience with sample content.

Integrations & QR Check-in API

ConnectHub+ exposes a small, purpose-built HTTP API so that a client organization can perform attendee check-in from **its own software** — typically an in-house QR scanning app, a turnstile controller, a kiosk, or an access-control system already deployed at the venue.

The integration is deliberately narrow: the external system reads a QR code and sends its content to ConnectHub+; ConnectHub+ answers with a screen-ready verdict (**access allowed / already checked in / denied**), together with the attendee's name and the event. On the ConnectHub+ side, attendance is recorded and the attendance certificate is issued automatically — the external system does not need to know anything about certificates, registrations, or payments.

1. Where the QR code comes from

Every registration to an event produces a single, unique ticket token. The same token is what gets scanned, no matter how the attendee presents it:

- **Attendee mobile app** — under *My Tickets*, each ticket renders its QR code on screen. This is the normal path for attendees who registered through the app.
- **Printed badges** — the panel can generate and print badge sheets for an event (name, event title and the check-in QR). This covers walk-in desks, lanyard badges and attendees who prefer paper.

Both surfaces encode **the same token**, so a badge and a phone screen are interchangeable at the door. The token format is:

```
TCK-XXXX-XXXX
```

It uses a restricted alphabet that omits visually ambiguous characters (no 0 / O , no 1 / I), which makes it safe to read aloud or type in by hand when a camera fails.

The token is a secret credential for that ticket. It should be treated like a boarding pass: fine to display to its owner, never to be published or logged in shared systems.

2. Base URL and content type

Item	Value
Base URL	<code>https://connecthub.fourstacklabs.com/api</code>
Format	JSON (<code>Content-Type: application/json</code>)
Transport	HTTPS only
Authentication	<code>X-API-Key</code> header

Two endpoints are available to integrators:

Method	Path	Purpose
<code>POST</code>	<code>/integracion/checkin</code>	Confirm access — records attendance and issues the certificate
<code>POST</code>	<code>/integracion/verificar</code>	Look up a ticket without recording anything

3. Authentication: per-institution API keys

3.1 The model

Access is authenticated with an **API key**, sent on every request in the `X-API-Key` header:

```
X-API-Key: chk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

An `Authorization` header is also accepted, for clients whose HTTP stack makes custom headers awkward:

```
Authorization: ApiKey chk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
```

Every key belongs to **exactly one institution**. When a request arrives, ConnectHub+ resolves the key to its owning institution and scopes the whole operation to that institution's events. This is the isolation boundary of the integration (see 3.4).

Keys are prefixed `chk_` followed by a high-entropy random value, so they are easy to recognize in configuration files and easy to spot if they ever leak into a log.

3.2 Obtaining a key from the panel

Keys are created by an administrator, from the web panel:

1. Sign in to the ConnectHub+ panel with an account holding the **System** or **Administration** role.
2. Go to **Administration** → **Integrations**.
3. Click **Generate key**.
4. Give the key a descriptive name — the name is how you will recognize it later in the list (for example, `Main gate scanner` , `Registration desk – Hall B` , `Turnstile controller`).
5. The key value is displayed. **Copy it now**.

The full key value is shown exactly once. ConnectHub+ stores only a cryptographic hash of the key, plus a short non-secret prefix used to identify it in the interface. Nobody — including ConnectHub+ support — can retrieve the value afterwards. If it is lost, the key must be revoked and a new one generated.

The Integrations screen also embeds a short "How to integrate" panel with the endpoint, the header and copy-ready request/response examples, so the developer receiving the key gets the technical details in the same place.

3.3 Managing, revoking and regenerating keys

The Integrations screen lists every key issued for the institution, with:

Column	Meaning
Name	The label given at creation
Prefix	First characters of the key (non-secret), to identify which key is which
State	Active or Revoked
Created by	The administrator account that generated it
Created	Creation date and time
Last use	Timestamp of the most recent authenticated call — blank if never used
Uses	Total number of authenticated calls made with the key

Revoking. Click **Revoke** on the row and confirm. Revocation is immediate and permanent: the very next request presenting that key is rejected with `401` . A revoked key cannot be reactivated.

Regenerating / rotating. There is no in-place "regenerate" that reuses a key's identity — rotation is done by creating a new key and retiring the old one. The recommended zero-downtime sequence is:

1. Generate a new key (give it a name that makes the rotation obvious, e.g. `Main gate scanner – 2026`).
2. Deploy the new key to the scanner application.
3. Confirm on the panel that the **Last use** and **Uses** counters of the new key are advancing.
4. Revoke the old key.

Because both keys are valid simultaneously during steps 2–3, scanners can be updated one at a time.

The *Last use* and *Uses* columns double as a lightweight audit trail: an unexpected jump in usage, or usage outside event hours, is a signal that a key may have been copied.

Note on scope of the key. The integration key authenticates *check-in operations only*. It cannot read the event catalogue, modify events, list attendees, or create or revoke other keys. Key administration is available only from the panel, under an authenticated administrator session — an integration key can never manage keys, including its own.

3.4 Isolation between institutions

Each key resolves to a single owning institution, and ConnectHub+ verifies that the scanned ticket belongs to an event of **that same institution** before doing anything else.

This means:

- A key issued to Institution A can never check in an attendee of Institution B's event, even if a valid Institution B ticket is presented to it. The response is a plain `DENEGADO` with the reason `The ticket belongs to another institution`.
- The response never leaks details (attendee name, event title) about tickets outside the key's institution.
- A compromised key is contained: its blast radius is limited to marking attendance for that one institution's events. It cannot read data, cannot delete anything, and cannot reach any other tenant.

3.5 Authentication errors

HTTP	Body message	Cause
401	Missing API key (header X-API-Key)	No <code>X-API-Key</code> (and no <code>Authorization: ApiKey ...</code>) header was sent
401	Invalid or revoked API key	The key is wrong, malformed, or has been revoked
400	Validation error	<code>qrToken</code> missing, not a string, or longer than 200 characters; <code>idEvento</code> not an integer

Important: authentication and validation failures use HTTP error codes, but *business outcomes do not*. A denied ticket is a successful call — see below.

4. Check-in endpoint

```
POST /integracion/checkin
```

This is the endpoint the scanner calls when staff intends to let the person in. It records attendance and triggers certificate issuance.

4.1 Request

```
POST /integracion/checkin HTTP/1.1
Host: connecthub.fourstacklabs.com
X-API-Key: chk_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
Content-Type: application/json

{
  "qrToken": "TCK-UZFW-U47J",
  "idEvento": 262
}
```

Field	Type	Required	Description
<code>qrToken</code>	string (≤ 200 chars)	Yes	The content read from the QR code
<code>idEvento</code>	integer	No	Door gating: restrict this scanner to one specific event

About `qrToken` . Send the QR content exactly as your scanner produced it. ConnectHub+ normalizes it for you:

- A bare token (`TCK-UZFW-U47J`) works.
- A token embedded in a URL or in surrounding text works too — the token pattern is extracted automatically. This matters because many off-the-shelf scanning SDKs return a full URL rather than the raw payload.
- Case is irrelevant; leading and trailing whitespace is trimmed.

About `idEvento` . Omit it and the scanner accepts any valid ticket belonging to the institution — appropriate for a single-event venue or a general entrance. Supply it and the scanner becomes a door for that event only: a valid ticket for a *different* event of the same institution is rejected with a clear reason instead of being silently checked in at the wrong door. This is the recommended configuration when several events run in parallel in the same building.

4.2 Response — the three outcomes

All three outcomes return **HTTP 200**. The verdict lives in the `acceso` field, which is one of `PERMITIDO` , `YA_REGISTRADO` or `DENEGADO` . Client applications should branch on `acceso` , not on the status code.

A. `PERMITIDO` — access allowed (first scan)

Attendance has just been recorded and the certificate issued.

```
{
  "acceso": "PERMITIDO",
  "participante": {
    "nombre": "Francisco Andrade",
    "email": "f@example.com",
    "documento": "C0912345678"
  },
  "evento": {
    "id": 262,
    "titulo": "TECH INNOVATION SUMMIT 2026",
    "fecha": "2026-09-30"
  },
  "fechaIngreso": null,
  "certificado": "CERT-A1B2C3D4E5F6"
}
```

Field	Description
<code>participante.nombre</code>	Attendee's full name; falls back to the email address if no name is on file
<code>participante.email</code>	Attendee's email
<code>participante.documento</code>	Identity document (type + number), or <code>null</code> if not recorded
<code>evento.id</code> / <code>titulo</code> / <code>fecha</code>	Event identifier, title and date (<code>YYYY-MM-DD</code>)
<code>fechaIngreso</code>	<code>null</code> on this outcome — the entry is being recorded right now; the timestamp is stored server-side and returned on any subsequent scan
<code>certificado</code>	Code of the attendance certificate issued for this attendee, e.g. <code>CERT-A1B2C3D4E5F6</code>

B. `YA_REGISTRADO` — already checked in (repeat scan)

The ticket is valid, but this attendee has already entered. This is **informational, not a rejection** — ConnectHub+ deliberately does not decide whether to let the person through, because a second scan is legitimate in many situations (re-entry after stepping out, staff double-checking, a scanner retrying after a network glitch). Your staff makes the call; the API just tells them what it knows.

```
{
  "acceso": "YA_REGISTRADO",
  "participante": {
    "nombre": "Francisco Andrade",
    "email": "f@example.com",
    "documento": "C0912345678"
  },
  "evento": {
    "id": 262,
    "titulo": "TECH INNOVATION SUMMIT 2026",
    "fecha": "2026-09-30"
  },
  "fechaIngreso": "2026-09-30 09:12"
}
```

`fechaIngreso` carries the timestamp of the **original** entry (`YYYY-MM-DD HH:MM`), which is what staff needs on screen to judge the situation. No `certificado` field is returned, because the certificate was already issued on the first scan.

C. `DENEGADO` — denied

```
{
  "acceso": "DENEGADO",
  "motivo": "Ticket not found"
}
```

`motivo` explains the rejection. The possible values are:

motivo	Meaning
<code>Ticket not found</code>	The token does not correspond to any registration — an invalid, mistyped, damaged or fabricated QR
<code>The ticket belongs to another institution</code>	A real ticket, but for an event owned by a different institution than the API key
<code>The ticket is for a different event</code>	Only when <code>idEvento</code> was sent: a valid ticket of your institution, but for a different event

For the third case only, the response also includes `participante` and `evento` , so the scanner can show staff *which* event the person actually holds a ticket for and redirect them to the correct door:

```
{
  "acceso": "DENEGADO",
  "motivo": "The ticket is for a different event",
  "participante": {
    "nombre": "Francisco Andrade",
    "email": "f@example.com",
    "documento": "C0912345678"
  },
  "evento": {
    "id": 271,
    "titulo": "AI WORKSHOP - TRACK B",
    "fecha": "2026-09-30"
  }
}
```

4.3 Suggested on-screen treatment

acceso	Display
PERMITIDO	Green — "Access confirmed" + attendee name and event title
YA_REGISTRADO	Amber — "Already checked in at {fechaIngreso}" + attendee name
DENEGADO	Red — the <code>motivo</code> , plus the event title when present

Use colour **and** text (and ideally a distinct sound per outcome). Door staff work fast, in poor light, often glancing rather than reading — the verdict should be unmistakable at arm's length.

5. Verification endpoint (look up without recording)

```
POST /integracion/verificar
```

Same authentication, same request body (`idEvento` is ignored here). This endpoint answers "is this ticket good?" **without** marking attendance and **without** issuing a certificate — it has no side effects at all.

Use it for pre-validation queues, information desks, informational displays, or any point where you want to inspect a ticket without consuming it.

Request

```
POST /integracion/verificar HTTP/1.1
X-API-Key: chk_XXXXXXXXXXXXXXXXXXXXXXXXXXXX
Content-Type: application/json

{ "qrToken": "TCK-UZFW-U47J" }
```

Response — valid ticket

```
{
  "valido": true,
  "yaRegistrado": false,
  "fechaIngreso": null,
  "participante": {
    "nombre": "Francisco Andrade",
    "email": "f@example.com",
    "documento": "C0912345678"
  },
  "evento": {
    "id": 262,
    "titulo": "TECH INNOVATION SUMMIT 2026",
    "fecha": "2026-09-30"
  }
}
```

`yaRegistrado` tells you whether this ticket has already been checked in, and `fechaIngreso` carries the entry timestamp when it has.

Response — invalid ticket

```
{
  "valido": false,
  "motivo": "Ticket not found"
}
```

The `motivo` values are the same as for check-in (excluding the event-mismatch reason, which only applies when gating by `idEvento` on check-in).

Verification is a read; check-in is the write. If your workflow verifies first and then checks in, remember that the check-in call is still the one that counts — never treat a successful `verificar` as attendance.

6. curl examples

Check-in

```
curl -X POST https://connecthub.fourstacklabs.com/api/integracion/checkin \
-H "X-API-Key: chk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX" \
-H "Content-Type: application/json" \
-d '{"qrToken":"TCK-UZFW-U47J"}'
```

Check-in gated to one event

```
curl -X POST https://connecthub.fourstacklabs.com/api/integracion/checkin \
-H "X-API-Key: chk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX" \
-H "Content-Type: application/json" \
-d '{"qrToken":"TCK-UZFW-U47J","idEvento":262}'
```

Verify without recording

```
curl -X POST https://connecthub.fourstacklabs.com/api/integracion/verificar \
-H "X-API-Key: chk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX" \
-H "Content-Type: application/json" \
-d '{"qrToken":"TCK-UZFW-U47J"}'
```

7. Idempotent behaviour

Sending the same `qrToken` to `/integracion/checkin` more than once is safe. The first successful call marks attendance and stamps the entry time; every subsequent call detects that the ticket is already checked in and returns `YA_REGISTRADO` with the *original* timestamp. Specifically, repeat calls:

- do **not** create a duplicate attendance record;
- do **not** overwrite or advance the recorded entry time;
- do **not** issue a second certificate;
- do **not** return an error.

This property is what makes the API safe for unreliable venue networks. If a request times out, or the connection drops before the response arrives, your application cannot know whether the check-in landed — and it does not need to. Simply retry the same token. Either it was never recorded (you get `PERMITIDO`) or it was (you get `YA_REGISTRADO` with the true entry time). Both are correct answers, and neither corrupts anything.

The same guarantee lets you build an **offline queue**: if the venue network fails, keep scanning, store the tokens locally, and replay them when connectivity returns. Replaying a token that was already flushed is harmless.

Certificate issuance is idempotent on the same basis — one attendance certificate per attendee per event, no matter how many times the ticket is scanned.

8. Automatic certificate issuance

Check-in is the trigger for attendance certification. On the first successful check-in of a ticket, ConnectHub+:

1. Marks the attendee as present and records the entry timestamp.
2. Issues an attendance certificate for that attendee and event.
3. Returns the certificate code in the `certificado` field of the response (format `CERT-` followed by a hexadecimal identifier, e.g. `CERT-A1B2C3D4E5F6`).

From that moment the certificate is available to the attendee inside the ConnectHub+ app, and the code can be used for public verification of the certificate's authenticity. The external scanner does not have to do anything with the code — it is returned mainly for receipt-printing, logging and support purposes. Displaying it on screen is optional.

The certificate carries the attendee's name, the event title and the issuing institution, rendered on the certificate template configured for that event in the panel.

Because certificates are driven by check-in, the completeness of your check-in operation directly determines who gets certified. Attendees who are waved through without a scan will not receive a certificate.

9. Best practices

9.1 Handling the network

- **Always retry on failure.** Timeouts, 5xx responses and dropped connections should be retried with the same `qrToken`. The endpoint's idempotency makes this safe by design.
- **Use exponential backoff** (e.g. 1s, 2s, 4s, 8s, capped) rather than tight retry loops, so a brief backend hiccup does not turn into a stampede from every scanner at once.
- **Set a short client timeout** (2–5 seconds is usually right for a door) and show staff a clear "retrying" state instead of freezing the UI.
- **Queue offline scans.** Persist unsent tokens to local storage, flush them when connectivity returns, and surface the queue depth to staff so they know the device has pending work.
- **Never invent a local verdict.** If your app has not received a response, do not display `PERMITIDO`. Show a pending state and let staff decide; the door policy during an outage is an operational decision, not

something the scanner should fake.

- **Deduplicate rapid re-reads.** Cameras commonly read the same code several times per second. Debounce locally (e.g. ignore the same token for 3–5 seconds) to avoid needless traffic — correctness is already guaranteed, but the noise pollutes your own logs and the usage counters.

9.2 Protecting the key

- **Treat the key as a production credential.** Store it in a secrets manager, a secure keystore, or the platform's encrypted credential store — never in source control, never in a plain-text config committed to a repository, never in a screenshot or support ticket.
- **Avoid shipping the key inside a widely distributed mobile app.** Application binaries can be decompiled. If your scanner is a public app store download, route check-in calls through your own backend, which holds the key server-side. If the app is distributed privately to staff devices only, provision the key at first login rather than baking it into the build.
- **Issue one key per deployment,** not one key for everything: a key per scanner fleet, per venue, or per vendor. Granular keys mean a single compromise or a departing contractor costs you one revocation, not a full re-deployment.
- **Name keys meaningfully.** The name is the only thing distinguishing them in the panel once the value is hidden. `Registration desk - Hall B` beats `key2`.
- **Never send the key in a URL.** It belongs in the header only. Query strings end up in proxy logs, browser history and analytics.
- **Do not log the key,** and scrub it from crash reports and HTTP debug traces. If you must correlate, log the `chk_` prefix only.
- **Send only over HTTPS.** Do not disable certificate validation, even on a venue network with a captive portal.
- **Rotate periodically** — a sensible default is once per event season, and immediately upon any staff change with key access.
- **Revoke immediately** if a device is lost or stolen, a contractor's engagement ends, or the key may have been exposed. Revocation takes effect on the next request; then issue a replacement.
- **Watch the usage counters.** Review *Last use* and *Uses* on the Integrations screen after each event. Usage from a key that should be idle, or activity outside event hours, warrants investigation.

9.3 Operational tips

- **Gate scanners with `idEvento`** whenever multiple events run concurrently at the same site. It converts a confusing "wrong room" situation into an explicit, actionable message.
- **Show the attendee's name on screen** on every outcome that carries one. It is the fastest way for staff to catch a mismatch between the badge and the person holding it.
- **Provide a manual token entry fallback.** The token alphabet excludes ambiguous characters precisely so that a damaged badge can be typed in by hand when the camera cannot read it.
- **Test against a real ticket before the event.** Generate a registration, open its QR in the attendee app, scan it with `verificar` (which records nothing), and confirm the response renders correctly in your UI.

Reserve `checkin` for the live rehearsal.